MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión C del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_flores_diego
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/flores_diego/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código

¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

Tu respuesta

```
Es agarrar un objeto de la vida real, pero como atributos se pueden ignorar cosas, al igual
que de metodos, por ejemplo un carro, tomamos el nivel de gasolina, pero podemos ignorar el
color.
```

Corrección. El ejemplo del carro es válido: conserva el nivel de gasolina y omite el color. Falta decir el criterio: abstraer es quedarse con lo esencial para el problema que se resuelve, y el color se omite porque no interviene en ese contexto.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

Tu respuesta

```
sirve para crear un nuevo objeto, se declara con la primer letra mayuscula y se pasan los
atributos, y para referirse a ellos se usa el this y tambien se ocupa el new.
```

Corrección. Es correcto que el constructor se usa al crear el objeto con new. La distinción está incompleta: lleva el mismo nombre que la clase (no basta con que empiece con mayúscula) y no declara tipo de retorno, ni siquiera void; la idea central es que inicializa el estado de la instancia.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué impide final en una variable? Si guarda una referencia a un arreglo, ¿impide cambiar sus elementos?

Tu respuesta

```
Con final el contenido se mantiene igual en todo el programa, y no se puede cambiar el
contenido en ningun momento.
```

Corrección. Es correcto que final impide volver a asignar la variable. No responde la pregunta del arreglo y, al decir que el contenido no puede cambiar, da a entender lo contrario de lo correcto: final no permite cambiar la referencia, pero los elementos del arreglo sí se pueden modificar.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Por qué == no basta para comparar el contenido de dos String? Explica qué usarías para comparar sus textos.

Tu respuesta

```
== no diferencia entre espacios y mayusculas,y compara direccion de memoria. para comparar
strings se usa equals ( ) que compara el contenido de los strings.
```

Corrección. La idea central está bien: == compara la dirección (identidad) y equals compara el contenido. Lo de que == 'no diferencia espacios y mayúsculas' es incorrecto, porque == no compara el texto en absoluto.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen while y do-while? Explica qué ocurre si la condición es falsa desde el inicio.

Tu respuesta

```
En while si la condicion falla desde el inicio no entra al contenido, y con do- while entra al
menos una vez y despues verifica la condicion, si cumple sigue si no se sale.
```

Corrección. Correcta y completa: while revisa antes y puede hacer cero vueltas; do-while ejecuta al menos una vez y después revisa la condición.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[][] tabla = {{9, 4, 7}, {2, 6, 8}};
int[] fila = tabla[0];
int total = 0;
for (int i = 0; i < fila.length; i++) {
    fila[i] = fila[i] / 2;
```

```
    total += tabla[0][i];
    System.out.println(i + ":" + total);
}
System.out.println((double) (total / 2));
System.out.println("P" + fila[0] + tabla[1][0]);
```

Tu respuesta

```
0 : 4 2 3
1 : 5 5 7
5.0 5.0 7.0
P 9 4 7 2
```

Corrección. La salida correcta es 0:4 / 1:6 / 2:9 / 4.0 / P42. Solo el inicio de la primera línea (0:4) y el prefijo P de la última son reconocibles; la respuesta tiene cuatro renglones con valores de más. No se toma en cuenta que fila es un alias de tabla[0] (queda {4,2,3}), ni que total/2 es división entera antes del cast, ni que la última línea concatena fila[0] y tabla[1][0].

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] datos = {3, 6};
int total = 0;
for (int i = 0; i <= datos.length; i++)
    total += datos[i];
System.out.println(total);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
truena por el <=, solo se ocuparia <
```

Corrección. Correcto: compila pero truena, porque con <= el ciclo intenta leer datos[2], fuera del arreglo; debía usarse <.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer piezas = 6;
System.out.println(piezas + 2);
System.out.println("P" + piezas);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
8
p6
```

Corrección. La clasificación y la salida (8 / P6) son correctas. Falta explicar la regla: en piezas + 2 el Integer se desempaqueta y se suma, y con "P" se concatena como texto.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
static int medir(int valor) {
    if (valor > 0) return valor;
}
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
faltan las llaves en el cuerpo del if { }
```

Corrección. La clasificación es correcta (no compila), pero la causa no: las llaves en un if de una sola instrucción son opcionales. No compila porque falta un return cuando valor es cero o negativo, y Java exige que todos los caminos devuelvan un int.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada lámpara tiene etiqueta fija y nace apagada. Cuenta las lámparas creadas. Escribe encender() y una consulta del estado; cada lámpara cambia por separado. Usa el enum dado.

```
enum Estado { APAGADO, ENCENDIDO, PARPADEO }
```

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué tipo tiene el atributo de estado y con qué valor nace?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
public class Lampara{
Static int NUMLAMPS;
//Atributos
String nombreLampara;
String estado ;
//constructor
   Lampara (String NombreLampara, String EStado){
    this.NombreLampara = NombreLampara;
    this.estado = APAGADO;
    NUMLAMPS ++;
}
//metodos
String Cambio (char encender){
 if ( encender == 'S'){
    estado = ENCENDIDO;
    }else if(encender == 'P'){
    estado = PARPADEO;
    }else{
    estado = APAGADO;}
return estado;
}
String ConsultaEstado(){
return estado;
}
int Lamparas(){
return NUMLAMPS;
}
}
```

Justificaciones

```
estado tiene un tipo de dato string y no lo inicializo.
Declare una variable de tipo string para nombrar a la lampara, tambien uno int para contar el
numero de lamparas creadas. Al constructor le paso el nombre y el estado lo inicializo como
APAGADO para todas las lamparas.
Cambio devuelve un tipo string y recibe un char.
```

```
Consulta estado devuelve igual un string pero no recibe parametros y por ultimo Lamparas no
recibe parametros y devuelve un int que es el contador final de las lamparas creadas
```

Corrección. Bien: el contador static, el atributo de nombre, el incremento en el constructor y una consulta que devuelve el estado. Fallas: estado es String y no el enum Estado (debía ser Estado y nacer en Estado.APAGADO), no hay constante final para la etiqueta fija, el constructor recibe un parámetro de estado que no usa, y Cambio usa if/else cuando se pedía un encender() sin estructuras de control. En el reto, el valor inicial APAGADO está bien, pero el tipo es incorrecto.

R4 · Código con repetición

Pregunta

Devuelve la posición de la menor temperatura usando for; si se repite, devuelve la primera. Los índices empiezan en 0. El arreglo no es null, tiene al menos un elemento y puede incluir negativos. No lo modifiques.

```
static int posicionMinima(int[] temperaturas)
```

Ejemplos (argumentos → retorno):

{8, 3, 5} → 1 · {7} → 0 · {-2, -8, -1} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int posicionMinima (int[] temperaturas) {
int menor = temperatura [0];
for (int i = 0; i < temperatura.length; i++) {
    if(menor < temperatura[i]{
    menor = temperatura[i];
    }
    return i;
}
```

Corrección. La cabecera y el uso del for están bien, y no se modifica el arreglo. Pero la comparación está invertida (menor < t[i] busca el mayor), se guarda el valor en lugar de la posición, y el return i dentro del ciclo termina en la primera vuelta (siempre devuelve 0 y falta un return después del ciclo). Lo correcto: guardar posicion = 0, recorrer desde 1, comparar t[i] < t[posicion] con < estricto, actualizar posicion = i y devolver posicion al final; además faltan un paréntesis y una llave, y se escribe temperatura en lugar de temperaturas.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Un equipo decide su mantenimiento según los ciclos de uso. Un valor menor o igual que 0 es "INVALIDO". Para los positivos, aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else. No imprimas ni leas del teclado.

```
static String mantenimiento(int ciclos)
```

Regla o valor recibido	Texto devuelto

Menor o igual que 0	"INVALIDO"
Divisible entre 24	"COMPLETO"
Si no, divisible entre 12	"REVISION"
Si no, divisible entre 4	"BASICO"
Cualquier otro	"NINGUNO"

Tu respuesta

```
static String mantenimiento(int ciclos) {
if (ciclos <= 0){
     return "INVALIDO";
}else if ( ciclos % 24 == 0){
    return "COMPLETO";
}else if (ciclos % 12 == 0){
    return "REVISION";
}else if (ciclos % 4 == 0){
    return "BASICO";
}else {
    return "NINGUNO";
    }
}
```

Corrección. Correcto y completo: primero valida ciclos <= 0, revisa 24 antes que 12 y 12 antes que 4, usa if / else-if / else y devuelve un valor en todos los caminos.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.